

Documentación: Sistema de Cifrado Híbrido (AES + RSA)

Autor: Sergio David Foster Medina

1. Explicación del funcionamiento del algoritmo híbrido

El sistema implementa un modelo de cifrado híbrido que combina la velocidad y eficiencia del cifrado simétrico (AES) con la seguridad en el intercambio de claves del cifrado asimétrico (RSA). El proceso consta de los siguientes pasos:

Proceso de Cifrado (`get_Msj_And_Key`):

1. Se genera una clave de sesión aleatoria para AES (32 bytes) y un Vector de Inicialización (IV) aleatorio (16 bytes) utilizando `os.urandom`.
2. El mensaje original se cifra con AES usando la clave de sesión y el IV generados.
3. Para asegurar la transmisión de los parámetros simétricos, la clave AES y el IV se cifran utilizando la clave pública RSA del receptor.
4. Se envían tres componentes: el mensaje cifrado (AES), la clave AES cifrada (RSA) y el IV cifrado (RSA).

Proceso de Descifrado (`decifrar_mensaje`):

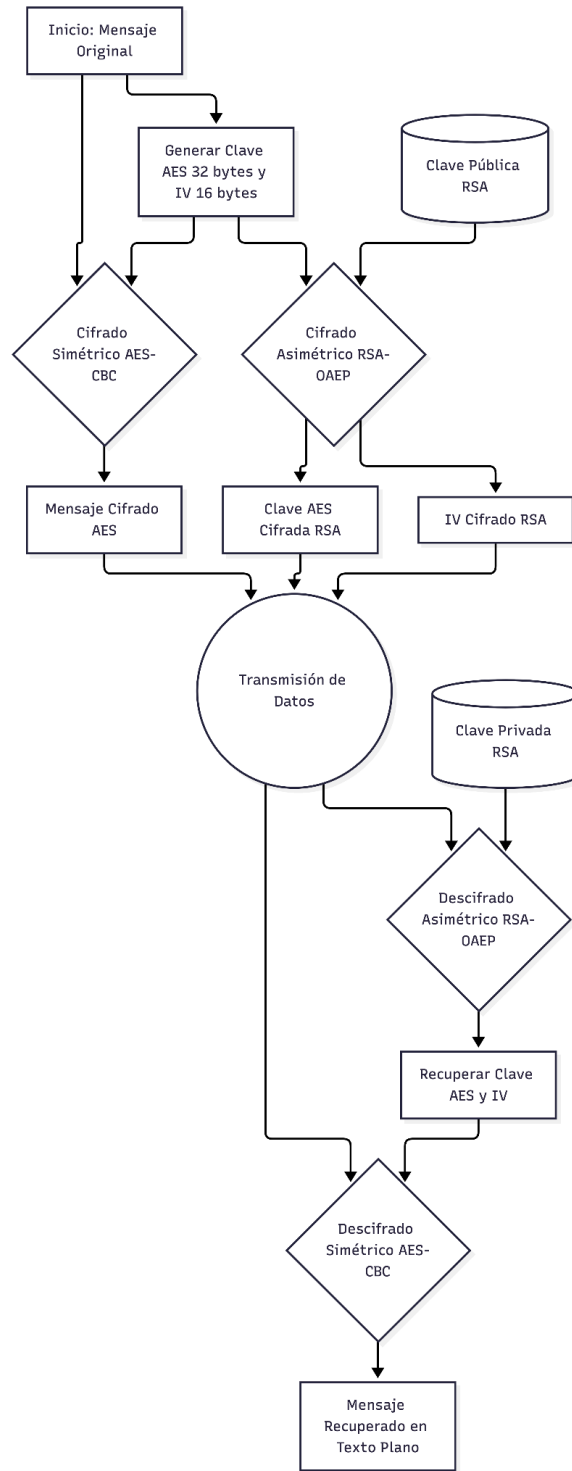
1. El receptor utiliza su clave privada RSA para descifrar el IV y la clave AES.
2. Con los parámetros simétricos recuperados, el receptor inicializa el descifrador AES y recupera el mensaje original en texto plano.

2. Justificación de elecciones de diseño

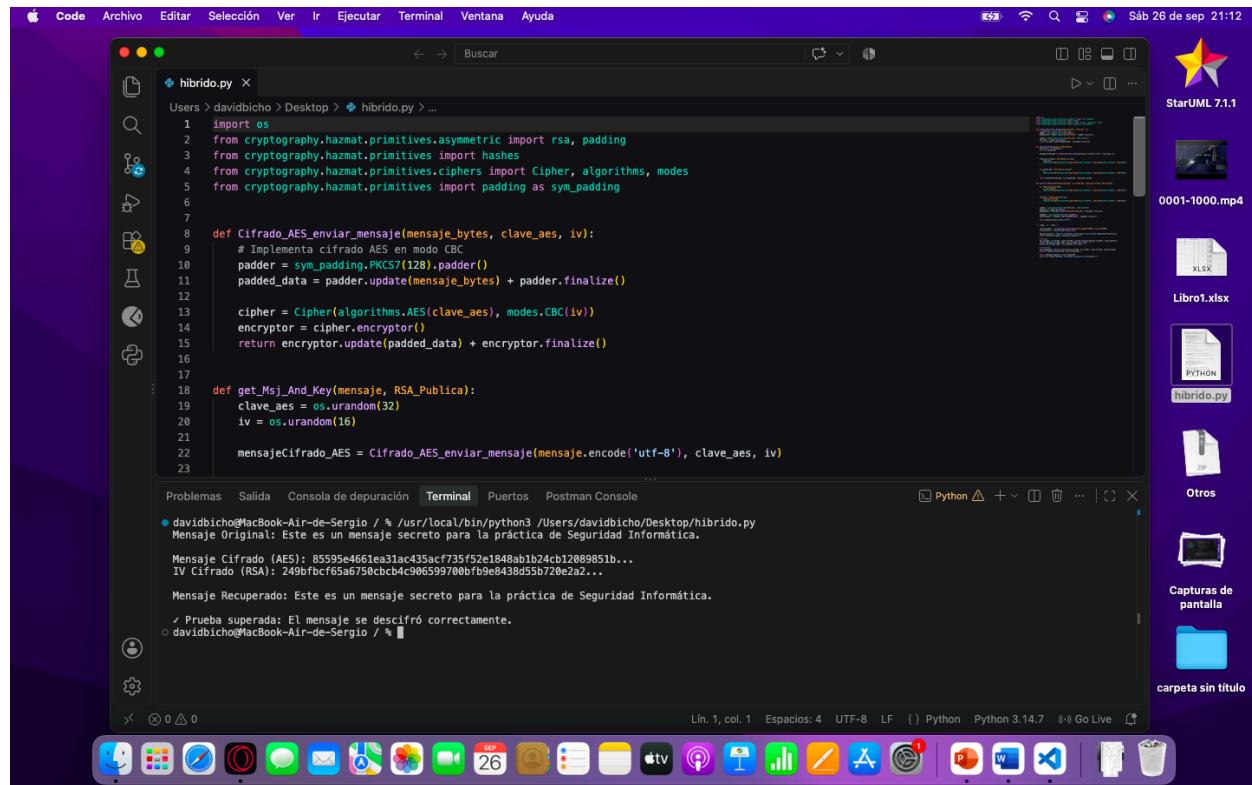
- **Algoritmo Simétrico (AES-256):** Se eligió una clave de 32 bytes (256 bits), lo cual proporciona el nivel máximo de seguridad en AES, haciéndolo resistente a ataques de fuerza bruta.
- **Modo de Operación (CBC):** El modo *Cipher Block Chaining* (CBC) garantiza que bloques de texto plano idénticos generen bloques de texto cifrado diferentes, ocultando patrones en los datos. Requiere un IV único para cada ejecución.
- **Padding Simétrico (PKCS7):** Dado que AES es un cifrado por bloques de 128 bits, PKCS7 asegura que el mensaje siempre sea un múltiplo exacto del tamaño del bloque antes de cifrar.
- **Algoritmo Asimétrico (RSA-2048):** Se generó una clave RSA con un tamaño de 2048 bits y un exponente público estándar de 65537. Este es el estándar actual recomendado, ofreciendo un balance óptimo entre seguridad y rendimiento.
- **Padding Asimétrico (OAEP + SHA256):** Para el cifrado RSA, se implementó *Optimal*

Asymmetric Encryption Padding (OAEP) utilizando la función hash SHA-256. Es crucial para prevenir ataques de texto cifrado elegido.

3. Diagrama de flujo del proceso



4. Evidencia y Código Fuente



The screenshot shows a macOS desktop environment. The main window is a code editor (VS Code) displaying a Python script named `hibrido.py`. The script implements a hybrid encryption system using AES-CBC and RSA. The code includes imports for `os`, `rsa`, `padding`, `hashes`, `Cipher`, `algorithms`, `modes`, and `padding` from the `cryptography.hazmat.primitives` module. It defines a function `Cifrado_AES_enviar_mensaje` for AES encryption, a function `get_Msj_And_Key` for key generation, and a function `mensajeCifrado_AES` for the final encryption process. The terminal window at the bottom shows the execution of the script, displaying the original message, the encrypted message (AES and RSA), and the recovered message, confirming successful decryption.

```
1 import os
2 from cryptography.hazmat.primitives.asymmetric import rsa, padding
3 from cryptography.hazmat.primitives import hashes
4 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
5 from cryptography.hazmat.primitives import padding as sym_padding
6
7
8 def Cifrado_AES_enviar_mensaje(mensaje_bytes, clave_aes, iv):
9     # Implementa cifrado AES en modo CBC
10    padder = sym_padding.PKCS7(128).padder()
11    padded_data = padder.update(mensaje_bytes) + padder.finalize()
12
13    cipher = Cipher(algorithms.AES(clave_aes), modes.CBC(iv))
14    encryptor = cipher.encryptor()
15    return encryptor.update(padded_data) + encryptor.finalize()
16
17
18 def get_Msj_And_Key(mensaje, RSA_Publica):
19     clave_aes = os.urandom(32)
20     iv = os.urandom(16)
21
22     mensajeCifrado_AES = Cifrado_AES_enviar_mensaje(mensaje.encode('utf-8'), clave_aes, iv)
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Terminal Output:

```
DavidBicho@MacBook-Air-de-Sergio: / % /usr/local/bin/python3 /Users/davidbicho/Desktop/hibrido.py
Mensaje Original: Este es un mensaje secreto para la práctica de Seguridad Informática.
Mensaje Cifrado (AES): 85595e4661ea31ac435ac735f52e1848ab1b24cb12089851b...
IV Cifrado (RSA): 249bfbcf65a6750cbcb4c906599700bfb9e8438d55b720e2a2...
Mensaje Recuperado: Este es un mensaje secreto para la práctica de Seguridad Informática.
✓ Prueba superada: El mensaje se descifró correctamente.
DavidBicho@MacBook-Air-de-Sergio: / %
```

Pruebas de correcto funcionamiento

Las pruebas están automatizadas en el bloque principal del script. Al ejecutar el código, el sistema realiza una demostración completa mostrando los datos cifrados y validando (mediante `assert`) que el texto recuperado sea idéntico al original.

5. Análisis de seguridad del sistema implementado

Fortalezas:

- **Confidencialidad robusta:** La combinación de AES-256 y RSA-2048 con OAEP protege los datos contra ataques criptográficos modernos.
- **Aleatoriedad garantizada:** El uso de `os.urandom` asegura entropía a nivel del sistema operativo, evitando claves predecibles.
- **Protección del IV:** Al cifrar también el IV con RSA, se añade una capa extra de opacidad.

Vulnerabilidades y áreas de mejora:

- **Falta de Autenticación e Integridad:** El esquema actual asegura que un atacante no pueda leer el mensaje, pero no garantiza que no pueda modificarlo. Al usar AES-CBC sin un Código de Autenticación de Mensajes, el sistema es teóricamente susceptible a

ataques de oráculo de padding.

- **Solución propuesta:** Cambiar el modo de operación a GCM (*Galois/Counter Mode*) o implementar una firma digital RSA sobre el mensaje cifrado para verificar la identidad del emisor y la inmutabilidad del mensaje.